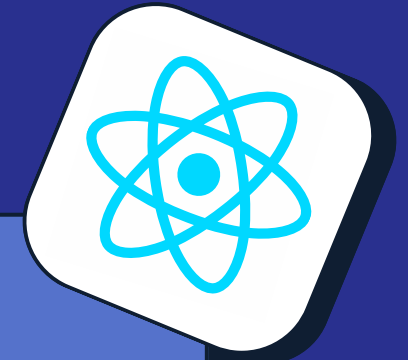




React Native

Buscando dados com APIs

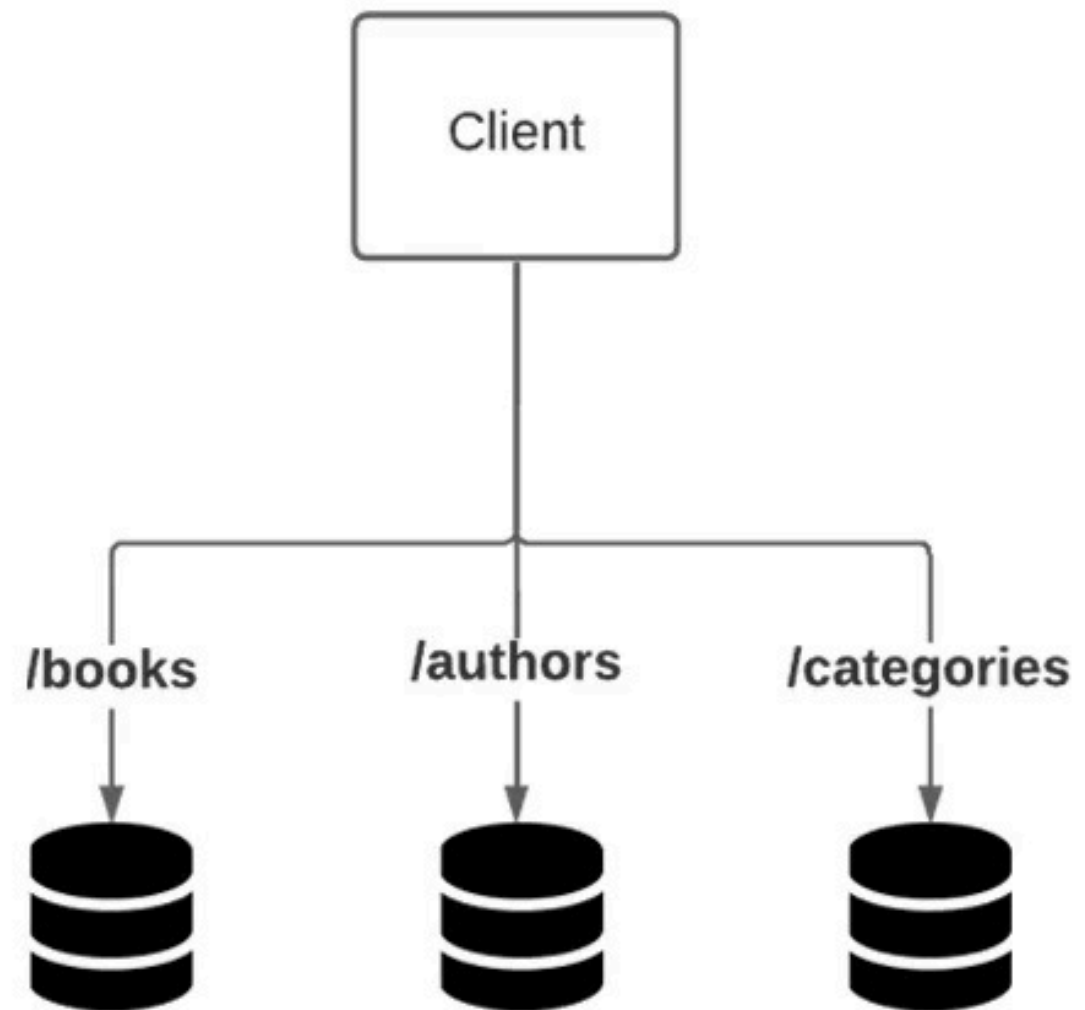
Introdução



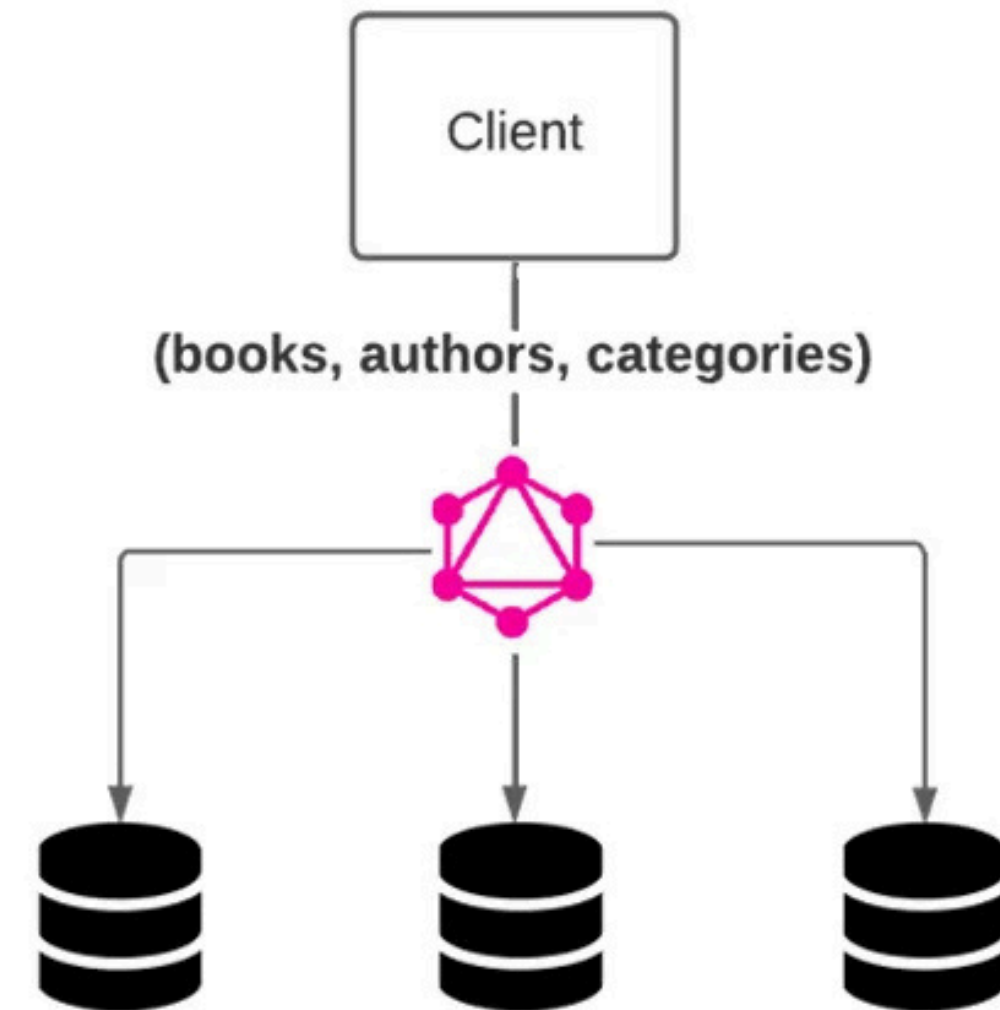
Uma **API** (Interface de Programação de Aplicações) é um conjunto de **definições** e **protocolos** que permitem a comunicação e **integração entre diferentes aplicações** de software. As APIs atuam como **mediadoras** entre usuários ou clientes e os recursos ou serviços web que eles desejam acessar. Elas permitem que as **organizações compartilhem recursos e informações, mantendo a segurança, o controle e a autenticação, ao determinar quem tem acesso a quê.**



REST API



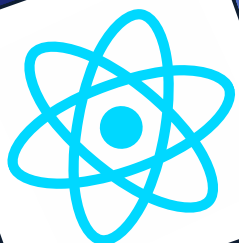
GraphQL API



Tipos de APIs

REST é um estilo arquitetural para sistemas de hipermídia em rede, desenvolvido no início dos anos 2000. Uma API que segue os princípios REST é chamada de **API RESTful**.

Características: As APIs RESTful são organizadas em torno de **recursos**, que são qualquer dado ou objeto que o cliente pode acessar e manipular. Cada recurso é identificado por uma **URI (Uniform Resource Identifier) única**. A comunicação ocorre de forma **stateless (sem estado)**, o que significa que cada requisição é independente e contém todas as informações necessárias para ser processada, sem depender de interações anteriores.



Client

```
fetch('https://api.exemplo.com/users/123')
  .then(response => {
    if (!response.ok) {
      throw new Error('Erro na rede');
    }
    return response.json();
  })
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error('Erro:', error);
  });
```

Server

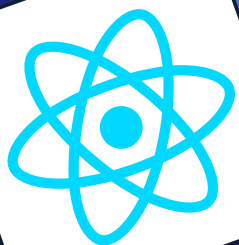
```
{
  "id": 123,
  "nome": "João Silva",
  "email": "joao.silva@exemplo.com",
  "idade": 30,
  "endereco": {
    "rua": "Rua das Flores",
    "numero": 45,
    "cidade": "São Paulo",
    "estado": "SP"
  }
}
```



Tipos de APIs

GraphQL é uma linguagem de consulta e tempo de execução de API, desenvolvida internamente pelo Facebook em 2012 e tornada de código aberto em 2015.

Características: o GraphQL expõe modelos de dados usando um único endpoint. Os clientes enviam consultas que especificam exatamente os dados que precisam, evitando o problema de excesso ou falta de busca de dados (over-fetching e under-fetching). É definida por um esquema que especifica os tipos de dados e suas relações.





Client

```
const query = `
  {
    usuario(id: "123") {
      id
      nome
      endereco {
        rua
        numero
      }
    }
  }
`;

fetch('https://api.exemplo.com/graphql', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({ query }),
})
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error('Erro:', error);
  });
```



Server

```
{
  "data": {
    "usuario": {
      "id": "123",
      "nome": "João Silva",
      "endereco": {
        "rua": "Rua das Flores",
        "numero": 45,
      }
    }
  }
}
```



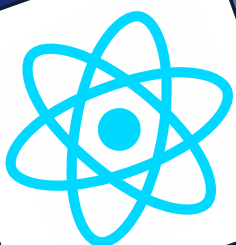
Princípios do REST

O **estilo arquitetural REST** define um conjunto de **seis restrições** que, quando aplicadas a um sistema, conferem propriedades desejáveis como **desempenho, escalabilidade, simplicidade, modificabilidade, visibilidade, portabilidade e confiabilidade**.

Uma API Web RESTful deve se alinhar a esses princípios:

Cliente/Servidor (Client/Server):

- Separação clara entre clientes e servidores, simplificando o servidor e melhorando a escalabilidade ao mover funcionalidades de interface para o cliente.



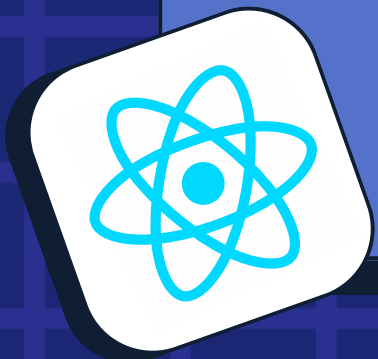
Princípios do REST

Sem estado (Stateless):

- Comunicação sem estado, onde o servidor não armazena dados do cliente entre requisições. Cada requisição deve conter todas as informações necessárias, mantendo o estado no cliente, o que melhora escalabilidade e confiabilidade.

Cache:

- Respostas da API devem indicar capacidade de cache, permitindo armazenamento e reutilização para melhorar desempenho e reduzir carga no servidor.



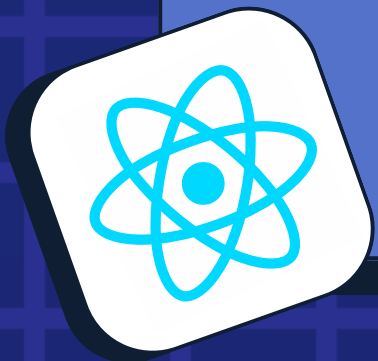
Princípios do REST

Interface uniforme (Uniform Interface):

- Identificação de recursos por URIs.
- Manipulação de recursos através de representações.
- Mensagens auto-descritivas.
- HATEOAS: uso de links para descobrir recursos dinamicamente.

Sistema em camadas (Layered System):

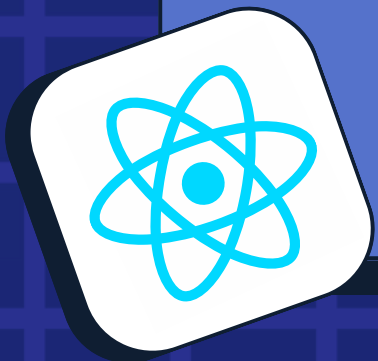
- O cliente não distingue se está conectado ao servidor final ou a um intermediário, permitindo a introdução de proxies para melhorar desempenho, segurança e encapsular sistemas legados.



Princípios do REST

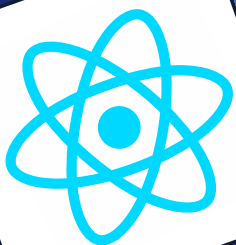
Código sob demanda (Code-on-demand) (opcional):

- Os servidores podem temporariamente estender ou personalizar a funcionalidade de um cliente transferindo lógica (como applets ou scripts) para o cliente, que pode ser executada em uma máquina virtual padrão.
- Esta é uma restrição opcional, pois, embora simplifique os clientes e melhore a extensibilidade do sistema, reduz a visibilidade.



Métodos HTTP (Verbos)

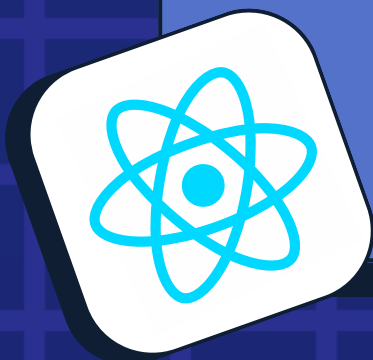
O protocolo **HTTP** (HyperText Transfer Protocol) é o conjunto de regras que permite a troca de informações entre clientes e servidores na web, sendo o "idioma" fundamental para a comunicação em aplicações web. Ele **define um conjunto de métodos** de requisição, também chamados de **verbos HTTP**, que indicam a ação a ser executada em um recurso.



Método HTTP (Verbo: GET)

GET

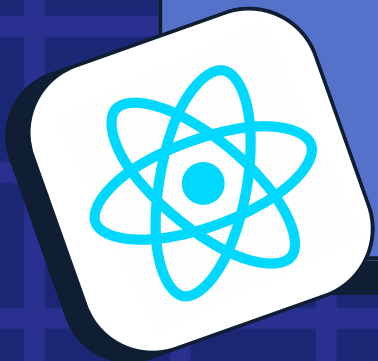
- **Objetivo:** Utilizado para **recuperar dados** de um recurso específico
- **Uso:** Os parâmetros são passados na URL (URI), o que facilita o cache e a indexação. Uma requisição GET deve retornar dados e não deve criar, atualizar ou deletar um recurso.



Método HTTP (Verbo: POST)

POST

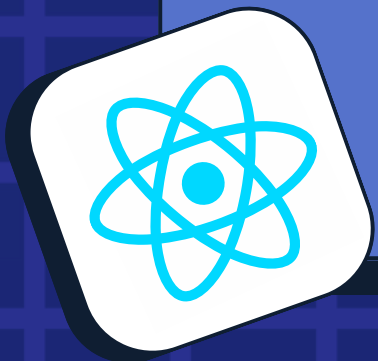
- **Objetivo:** Usado para enviar dados para o servidor, geralmente para criar um novo recurso.
- **Uso:** Os dados são enviados no corpo da requisição, permitindo o envio de informações mais complexas e sendo menos explícitos na URL.



Método HTTP (Verbo: PUT)

PUT

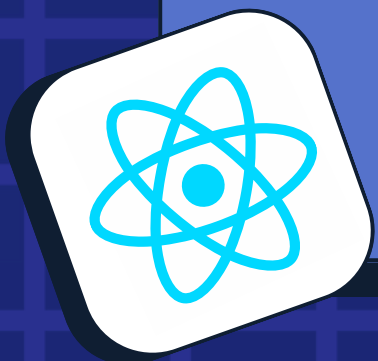
- **Objetivo:** Utilizado para atualizar um recurso existente completamente ou, em alguns casos, criar um novo recurso se ele não existir e a rota suportar.
- **Uso:** Os dados são enviados no corpo da requisição, substituindo completamente o recurso existente.



Método HTTP (Verbo: DELETE)

DELETE

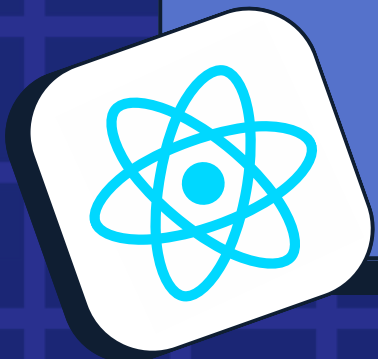
- **Objetivo:** Usado para remover um recurso específico do servidor.
- **Uso:** requisição não contém um corpo, e a URL especifica qual recurso deve ser excluído. O servidor deve processar a requisição e remover o recurso indicado.



Método HTTP (Verbo: PATCH)

PATCH

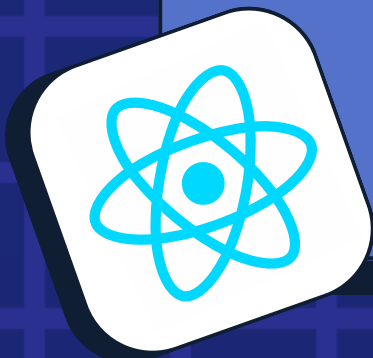
- **Objetivo:** Usado para aplicar modificações parciais a um recurso existente.
- **Uso:** Os dados são enviados no corpo da requisição, contendo apenas as alterações que devem ser aplicadas ao recurso. Isso permite atualizações mais eficientes, pois não é necessário enviar o recurso completo.



Códigos de Status HTTP

Categorias de Códigos de Status HTTP:

- 1xx (Respostas Informativas)
- 2xx (Respostas de Sucesso)
- 3xx (Respostas de Redirecionamento)
- 4xx (Respostas de Erro do Cliente)
- 5xx (Respostas de Erro do Servidor)

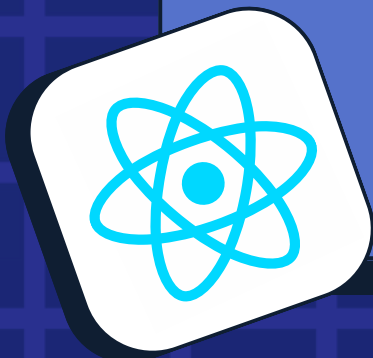


Autenticação

Autenticação: Confirma a identidade de um usuário ou aplicação.

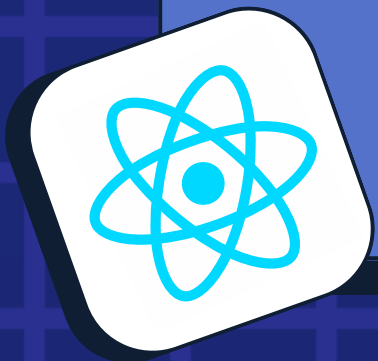
Métodos Comuns:

- Chave de API
- Token Bearer
- Autenticação Básica
- Cookies/Sessões
- Certificado Digital
- OAuth 2.0



CORS (Cross-Origin Resource Sharing)

CORS é um mecanismo que permite ou restringe solicitações de recursos de uma página web a partir de outro domínio. É fundamental para a segurança das aplicações web, evitando acessos não autorizados a dados.

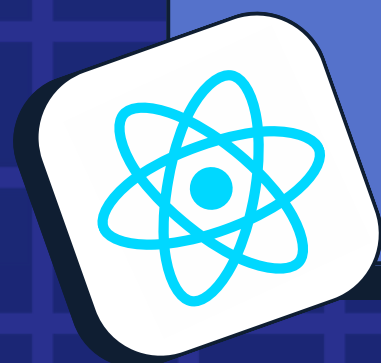


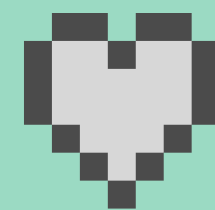
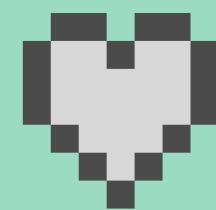
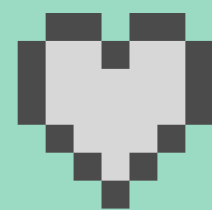
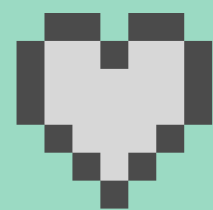
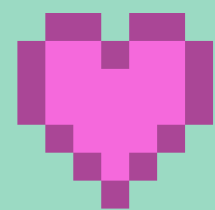
CORS (Cross-Origin Resource Sharing)

Função: Permite que diferentes partes, como APIs, negociem quais tipos de dados estão sendo enviados e aceitos, utilizando cabeçalhos HTTP como Content-Type e Accept.

Erros Comuns

- **Cabeçalhos Ausentes ou Incorretos:** Podem causar falhas na requisição.
- **Erro 415 Unsupported Media Type:** Ocorre quando o servidor se recusa a processar a requisição devido a metadados incorretos.





GAME OVER